

Programovací jazyky a překladače

Přednáška 7, poznámky

Jan Voráček

VŠPJ Jihlava, voracek@vspj.cz

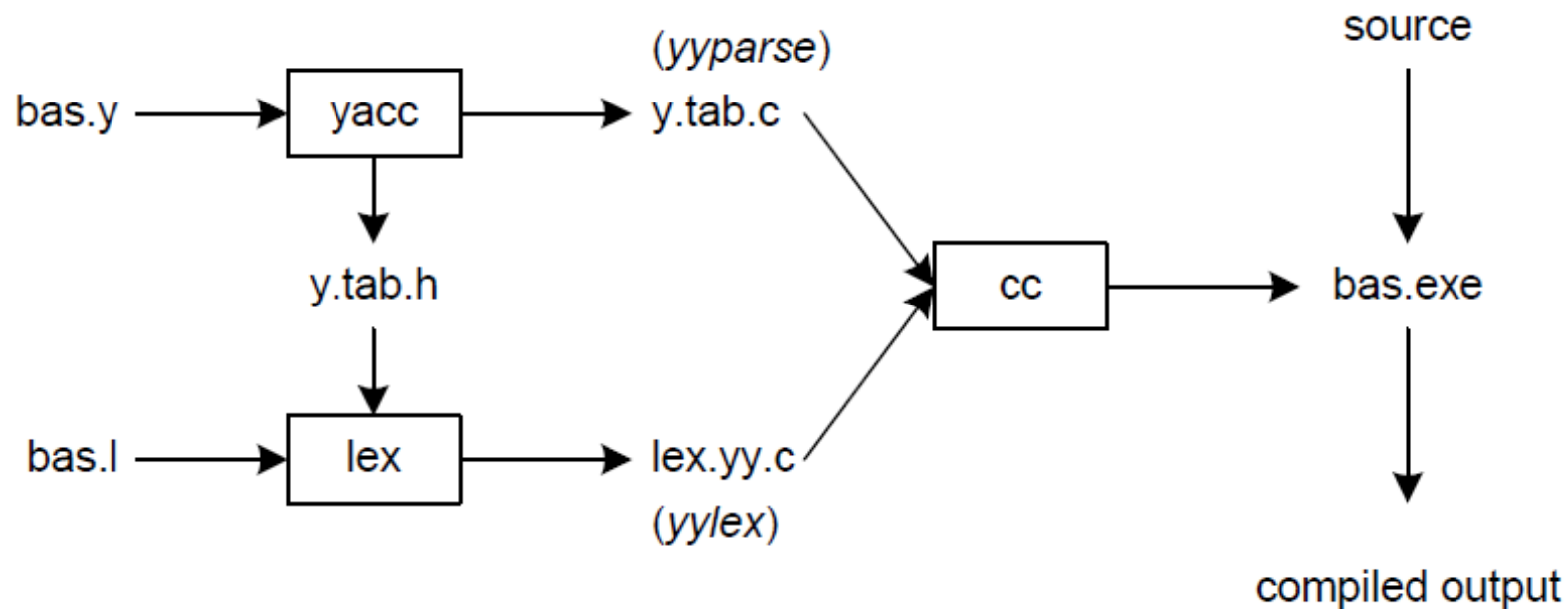
Dnešní témata

- Generátory syntaktických analyzátorů: *yacc/Bison*, část 1: SYNTAXE
 - Základy analýzy syntaxe
 - Syntaxe a sémantika zdrojových kódů
 - Spolupráce s lexikálním analyzátozem

Poznámka: jde o přípravu na dnešní cvičení. Teorie, související s LR překladem, bude probrána v následujících přednáškách

- Programová realizace LL(1) syntaktických analyzátorů (dokončení z minula)
 - Kontextové množiny FIRST a FOLLOW
 - Tabulková implementace LL syntaktických analyzátorů

Spolupráce *Lex* + *yacc*



Překlad:

```
yacc -d bas.y          # y.tab.h, y.tab.c
lex bas.l               # lex.yy.c
cc lex.yy.c y.tab.c -obas.exe # bas.exe
```

Spuštění a vnitřní volání funkcí:

main -> yyparse -> yylex

Důležité přepínače:

- d generuje sdílený soubor *y.tab.h*
- v vytvoří soubor *y.output* s podrobnými informacemi a procesu syntaktické analýzy, včetně charakteristiky případných konfliktů a způsobu jejich řešení
- t ve spolupráci s makrem *YYDEBUG* a na základě hodnoty proměnné *yydebug* přidá ladicí (trace, debug) informace do souboru *y.tab.c*

YACC = Yet Another Compiler

% {
Header

% }

%union

%token <...> *nebo* %left ...

%type <...>

alternativně



%token = %term

%%

Rules part

%%

User defined part

Konstruktory překladačů

Miroslav Beneš
Dušan Kolář



Část definice

```
%union {  
    float num;  
    char *id;  
};  
  
%start program  
%token <num> NUMBER  
%token <id> ID  
%token NEWLINE EQUALS PRINT  
%type <num> exp  
%type <num> stmt  
%left PLUS MINUS  
%left TIMES DIVIDE  
%left LPAREN RPAREN  
%nonassoc UMINUS
```

Část pravidla

```
exp: MINUS exp %prec UMINUS
```

- `yylval`: default INT
 - Lze předefinovat externí proměnnou `YYSTYPE` v `y.tab.h`

```
%union {int ival; double dval;}  
%token <ival> INDEX  
%token <dval> NUMBER  
%type <dval> expr  
%type <dval> mulex  
%type <dval> term
```

- Obsah `y.tab.h`

```
#define INDEX 258  
#define NUMBER 259  
typedef union {int ival; double dval;} YYSTYPE;  
extern YYSTYPE yylval;
```


LEX <-----> YACC

Jednotlivé znaky lze předávat:

A/ Jako tokeny:

```
"a"                return(a_TOK);
```

přičemž v *yacc_u* pak musí být definice

```
%token a_TOK
```

a v souboru *y.tab.h* jsou jim přiřazeny příslušné celočíselné identifikátory > 256

B/ Přímou jednoznakových gramatických elementů:

```
.                yytext[0];
```

a v *yacc_u* se pracuje s jejich konkrétní hodnotou, např. '+'

Deklarace, gramatika target -> A B C

```
%union {  
    struct node {  
        int type;  
        struct node *left;  
        struct node *right;  
    } *node;  
    int num;  
    char *str;  
}
```

```
%token<num> A B C
```

```
%type<str> target
```

Jednoznačnost gramatiky

Lze zajistit:

- Změnou gramatiky
- Změnou priority a asociativity **terminálů**
- Změnou priority a asociativity **pravidel**: namísto implicitně děděné priority a asociativity od **nejpravějšího terminálu** lze použít její redefinice pomocí %prec
 - V situaci $A \rightarrow \alpha ., a :$
 - Je-li priorita pravidla vyšší než priorita terminálu, redukuj
 - Jsou-li priority pravidla a terminálu stejné a asociativita pravidla je levá, redukuj
 - Jinak (v opačných případech) přesuň

Nejednoznačné gramatiky vedou na konflikty

Řešení konfliktů

Nelze-li konflikty vyřešit pomocí priority a asociativity, implicitní chování *yacc_u* je následující:

- shift-reduce (časté): **shift**
- reduce-reduce: **použije dřívější pravidlo ve specifikaci gramatiky a zároveň o tom podá zprávu.**

Pro účely ladění se používá přepínač **-v**, který generuje podrobný popis analýzy do souboru *y.output*

Konflikty nepatří mezi klasické programátorské chyby!

Výskyt chyb v překladu

- Lexikální i syntaktické analyzátory podporují i pokročilejší reakce na nalezené chyby, než je pouhé ukončení funkce. O situaci, kdy je chybný kód bez jakéhokoli varování přeložen do spustitelné podoby raději ani neuvažujeme.
- Obecné schéma zpracování chyb je následující:
 - Překladač najde chybu, **zotaví se** a pokračuje dál v činnosti (běžné)
 - Překladač najde chybu, **opraví ji** a pokračuje dál v činnosti (experimentální)
- **Lexikální chyby:** omezené možnosti
 - Ignorování vstupu až do nalezení dalšího jasného vzoru
 - Vrácení TOKEN_UNKNOWN a předání problému parseru
 - Samoopravné postupy – nedoporučují se

Zotavení po syntaktických chybách

Body zotavení jsou obvykle určeny množinou **klíčů**, tj. zotavení znamená vypuštění vstupního textu mezi dvěma syntakticky sousedícími klíči.

- Klíče musí být syntakticky jednoznačné.
- Čím je klíčů více, tím přesněji je chyba alokována a také se vypustí méně vstupního textu.

Analýza shora dolů:

- Nerekurzivní zotavení: hledají se ukončovací symboly typických konstrukcí (např. **)** pro výrazy nebo **;** pro příkazy) a ze zásobníku se pak odstraní vše, co s touto konstrukcí souvisí
- Hartmannovo schéma : dynamicky vytvářená množina klíčů na základě současného kontextu. Využívá obsah množiny SELECT (aktuální sada FIRST – FOLLOW).

Možnosti zpracování chyb v *yacc_u*

1. Chybové hlášení a **přerušení překladu** při syntaktické chybě:

```
void yyerror(char *s) { /*defaultně volaná při syntaktické chybě*/  
    fprintf(stderr, "line %d: %s\n", yylineno, s); }
```

- Funkce `yyerror` by měla být vždy implementována
- Je vhodné nějakým způsobem indikovat i číslo chybového řádku

2. **Zotavení** na úrovni alternativ pravidla pomocí sekvence:

<code>error</code>	speciální token
<code>yyclearin</code>	vyčištění chybné vstupní sekvence
<code>yyerrok</code>	nové nastavení detekce chyb

Zotavení uvnitř pravidla, příklad

A --> A 'b' 'c'

| A 'b' **error**

```
{ /* chybí 'c' */  
  printf("Line %d: Očekává se 'c'\n", yylineno); yyclearin; yyerrok;  
}
```

| **a error**

```
{ /* chybí 'b' */  
  printf("Line %d: Očekává se 'b'\n", yylineno); yyclearin; yyerrok;  
}
```

| **error**

```
{ /* problém s celým pravidlem */  
  printf("Line %d: Chyba v A\n", yylineno); yyclearin; yyerrok;  
}
```

;

Úkoly do 7. cvičení: ./parser3.y

(základní implementace)

```
%token a_TOK b_TOK c_TOK d_TOK e_TOK f_TOK g_TOK h_TOK
```

```
%%
```

```
S: A B      { printf("Correct derivation!\n"); };
```

```
A: a_TOK h_TOK | a_TOK b_TOK A c_TOK | a_TOK B c_TOK ;
```

```
B: C D E ;
```

```
C: d_TOK | e_TOK;
```

```
D: f_TOK D | f_TOK;
```

```
E: B | g_TOK ;
```

```
%%
```

Úkoly do 7. cvičení: ./parser4.y

(základní implementace)

```
S: A B      { printf("Correct derivation!\n"); };
```

```
A: 'a' 'h' | 'a' 'b' A 'c' | 'a' B 'c';
```

```
B: C D E ;
```

```
C: 'd' | 'e'
```

```
D: 'f' D | 'f';
```

```
E: B | 'g';
```

```
%%
```

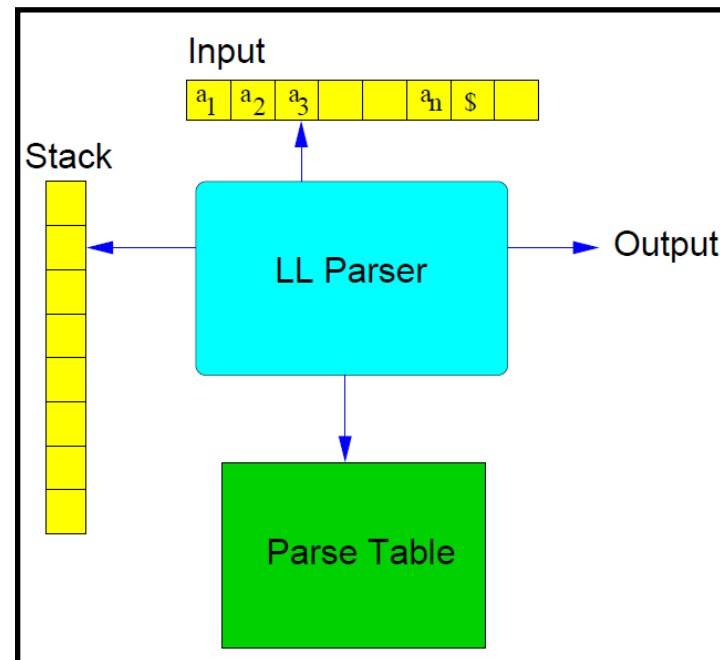
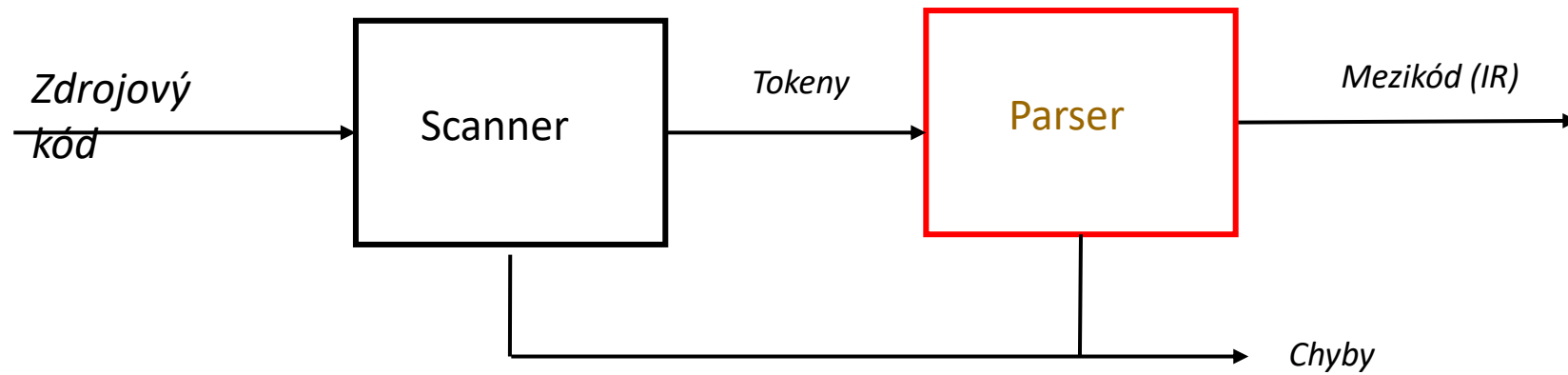
LL(1) gramatiky

Miroslav Beneš

Dušan Kolář



Tabulková implementace LL analyzátoru



Princip tabulkou řízeného překladače LL(1)

- Pro každé pravidlo $A \rightarrow \alpha$:
 - Pro každý terminál $\mathbf{a} \in \text{FIRST}(\alpha)$ přidej pravidlo $A \rightarrow \alpha$ na pozici tabulky $M[A, \mathbf{a}]$
 - Je-li $\varepsilon \in \text{FIRST}(\alpha)$, přidej pravidlo $A \rightarrow \alpha$ (respektive ε) na pozici tabulky $M[A, \mathbf{b}]$ pro každý terminál $\mathbf{b} \in \text{FOLLOW}(A)$.
 - Je-li $\varepsilon \in \text{FIRST}(\alpha)$ a $\text{EOF} \in \text{FOLLOW}(A)$, přidej pravidlo $A \rightarrow \alpha$ na pozici tabulky $M[A, \text{EOF}]$
- Rekurzivní analýza je pak simulována pomocí tabulky a zásobníku.
- Prediktivní analýze pracuje u LL gramatiky bez návratů, tj. deterministicky

1. Gramatika

$E \rightarrow E + E$

$| E * E$

$| (E)$

$| \mathbf{d}$



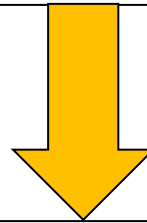
$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid \mathbf{d}$



$\text{FIRST}(+) = \{+\}$

$\text{FIRST}(*) = \{*\}$

$\text{FIRST}(\mathbf{d}) = \{\mathbf{d}\}$

$\text{FIRST}(() = \{($

$\text{FIRST}()) = \{)\}$

$\text{FIRST}(E) = \{(\mathbf{d}\}$

$\text{FIRST}(E') = \{\epsilon, +\}$

$\text{FIRST}(T) = \{(\mathbf{d}\}$

$\text{FIRST}(T') = \{\epsilon, *\}$

$\text{FIRST}(F) = \{(\mathbf{d}\}$

$\text{FOLLOW}(E) = \{\$,)\}$

$\text{FOLLOW}(E') = \{\$,)\}$

$\text{FOLLOW}(T) = \{+,), \$\}$

$\text{FOLLOW}(T') = \{+,), \$\}$

$\text{FOLLOW}(F) = \{*, +,), \$\}$

2. Překládová tabulka

$$\begin{aligned}
 E &\rightarrow TE' \\
 E' &\rightarrow +TE' \mid \epsilon \\
 T &\rightarrow FT' \\
 T' &\rightarrow *FT' \mid \epsilon \\
 F &\rightarrow (E) \mid \mathbf{d}
 \end{aligned}$$

FIRST(+)	= {+}	FIRST(E)	= {(, d }	FOLLOW(E)	= {\$,)}
FIRST(*)	= {*}	FIRST(E')	= { ϵ , +}	FOLLOW(E')	= {\$,)}
FIRST(d)	= { d }	FIRST(T)	= {(, d }	FOLLOW(T)	= {+,), \$}
FIRST((	= {(	FIRST(T')	= { ϵ , *}	FOLLOW(T')	= {+,), \$}
FIRST())	= {)}	FIRST(F)	= {(, d }	FOLLOW(F)	= {*, +,), \$}

Top of Stack	Input Symbol					
	d	+	*	(	)	\$
<i>E</i>	<i>E</i> → <i>TE'</i>			<i>E</i> → <i>TE'</i>		
<i>E'</i>		<i>E'</i> → + <i>TE'</i>			<i>E'</i> → ϵ	<i>E'</i> → ϵ
<i>T</i>	<i>T</i> → <i>FT'</i>			<i>T</i> → <i>FT'</i>		
<i>T'</i>		<i>T'</i> → ϵ	<i>T'</i> → * <i>FT'</i>		<i>T'</i> → ϵ	<i>T'</i> → ϵ
<i>F</i>	<i>F</i> → d			<i>F</i> → (<i>E</i>)		

3. Vzorový vstup 1: $d + d * d$

Top of Stack	Input Symbol					
	d	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow d$			$F \rightarrow (E)$		

Stack	Input	Rule
$\$ E$	$d + d * d \$$	$E \rightarrow TE'$
$\$ E'T$	$d + d * d \$$	$T \rightarrow FT'$
$\$ E'T'F$	$d + d * d \$$	$F \rightarrow d$
$\$ E'T'd$	$d + d * d \$$	
$\$ E'T'$	$+ d * d \$$	$T' \rightarrow \epsilon$
$\$ E'$	$+ d * d \$$	$E' \rightarrow +TE'$
$\$ E'T+$	$+ d * d \$$	
$\$ E'T$	$d * d \$$	$T \rightarrow FT'$
$\$ E'T'F$	$d * d \$$	$F \rightarrow d$
$\$ E'T'd$	$d * d \$$	
$\$ E'T'$	$* d \$$	$T' \rightarrow *FT'$
$\$ E'T'F*$	$* d \$$	
$\$ E'T'F$	$d \$$	$F \rightarrow d$
$\$ E'T'd$	$d \$$	
$\$ E'T'$	$\$$	$T' \rightarrow \epsilon$
$\$ E'$	$\$$	$E' \rightarrow \epsilon$
$\$$	$\$$	Accept

4. Vzorový vstup 2: $(d + d) * d$

Top of Stack	Input Symbol					
	d	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow d$			$F \rightarrow (E)$		

Stack	Input	Rule
\$ E	(d + d) * d\$	$E \rightarrow TE'$
\$ E' T	(d + d) * d\$	$T \rightarrow FT'$
\$ E' T' F	(d + d) * d\$	$F \rightarrow (E)$
\$ E' T') E (	(d + d) * d\$	
\$ E' T') E	d + d) * d\$	$E \rightarrow TE'$
\$ E' T') E' T	d + d) * d\$	$T \rightarrow FT'$
\$ E' T') E' T' F	d + d) * d\$	$F \rightarrow d$
\$ E' T') E' T' d	d + d) * d\$	
\$ E' T') E' T'	+ d) * d\$	$T' \rightarrow \epsilon$
\$ E' T') E'	+ d) * d\$	$E' \rightarrow +TE'$
\$ E' T') E' T +	+ d) * d\$	
\$ E' T') E' T	d) * d\$	$T \rightarrow FT'$
\$ E' T') E' T' F	d) * d\$	$F \rightarrow d$
\$ E' T') E' T' d	d) * d\$	
\$ E' T') E' T'	) * d\$	$T' \rightarrow \epsilon$
\$ E' T') E'	) * d\$	$E' \rightarrow \epsilon$
\$ E' T')	) * d\$	
\$ E' T'	* d\$	$T' \rightarrow *FT'$
\$ E' T' F *	* d\$	
\$ E' T' F	d\$	$F \rightarrow d$
\$ E' T' d	d\$	
\$ E' T'	\$	$T' \rightarrow \epsilon$
\$ E'	\$	$E' \rightarrow \epsilon$
\$	\$	Accept

Konstrukce LL překladačové tabulky pro obecnou gramatiku

1. Gramatika:

$S \rightarrow Bc \mid DB$

$B \rightarrow ab \mid cS$

$D \rightarrow d \mid \varepsilon$

2. Množiny FIRST a FOLLOW:

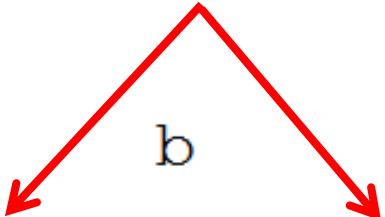
X	FIRST(X)	FOLLOW(X)
D	{ d, ε }	{ a, c }
B	{ a, c }	{ c, EOF }
S	{ a, c, d }	{ c, EOF }
Bc	{ a, c }	
DB	{ d, a, c }	
ab	{ a }	
cS	{ c }	
d	{ d }	
ε	{ ε }	

Konstrukce LL překladačové tabulky

2

3. Tabulka

KONFLIKT



	a	b	c	d	EOF
S	B c D B		B c D B	D B	
B	ab		cS		
D	epsil		epsil	d	

$S \rightarrow Bc \mid DB$

$B \rightarrow ab \mid cS$

$D \rightarrow d \mid \varepsilon$

X	FIRST(X)	FOLLOW(X)
D	{ d, ε }	{ a, c }
B	{ a, c }	{ c, EOF }
S	{ a, c, d }	{ c, EOF }
Bc	{ a, c }	
DB	{ d, a, c }	
ab	{ a }	
cS	{ c }	
d	{ d }	
ε	{ ε }	

Zotavení z chyby pro tabulkové LL překladače

- **Chyba:** při překladu přejdeme na prázdné pole v tabulce
- **Konflikt:** v jednom poli je více pravidel (gramatika není LL)
- **Zotavení** = pokračování v překladu po chybě/konfliktu
 - **Na úrovni tabulky:** speciální obslužná rutina pro každé prázdné pole
 - **Panic mode:** překladač pracuje se vstupem a/nebo zásobníkem tak dlouho, dokud se nesynchronizuje
 - Definuje se množina synchronizačních tokenů (ST)
 - ST = např. středník, uzavírací závorky, vybraná klíčová slova
 - V případě chyby a přítomnosti neterminálu na vrcholu zásobníku překladač vypouští vstup až do nalezení ST
 - V případě chyby a přítomnosti terminálu na vrcholu zásobníku vypustíme tento terminál z vrcholu zásobníku (předpokládáme, že úplně chybí, tj. byl zapomenut ve vstupu)

Interaktivní rozhraní pro práci s gramatikami 1

Enter a grammar:

```
S -> id
    | V assign E.
V -> id.
E -> V
    | num.
```

[View Vital Statistics](#)

Here's a small, quick, example grammar to give you an idea of the format of the grammars:

```
S -> id
    | V assign E.
V -> id.
E -> V
    | num.
```

To see more grammars and learn more about the format of the grammars:

- Read about the [structure of the grammars](#).
- Look at some [example grammars](#).



Grammar

The language is finite and the grammar is unambiguous.

Some sentences generated by this grammar: {id, id assign id, id assign num}

```
S -> id
    | V assign E.
V -> id.
E -> V
    | num.
```

- All nonterminals are reachable and realizable.
- There are no nullable nonterminals.
- The endable nonterminals are: V S E.
- No cycles.

nonterminal	first set	follow set	nullable	endable
S	id	$\emptyset$	no	yes
E	num id	$\emptyset$	no	yes
V	id	assign	no	yes

The grammar is not LL(1) because:

- S has a first set conflict.

- attempt to [transform](#) the grammar (to LL(1))
- generate [LL\(1\)](#) parsing table
- generate [LR\(0\)/SLR\(1\)](#) automaton
- generate [LALR\(1\)](#) automaton
- generate [LR\(1\)](#) automaton

Return home to [enter a new grammar](#).

Zdroj: <http://smlweb.cpsc.ucalgary.ca/start.html>

Interaktivní rozhraní pro práci s gramatikami 2

EditTransformAnalyze

Type a grammar here:
s -> id
| v assign E.
v -> id.
E -> v
| num.

Analysis

Sanity Checks

- All nonterminals are reachable.
- All nonterminals are realizable.
- The grammar contains no cycles.
- The grammar is null unambiguous.

Example Sentences

- id
- id assign num
- id assign id

Nonterminals

Symbol	Nullable?	Endable?	First set	Follow set
<i>S</i>		Endable	id	\$
<i>V</i>		Endable	id	assign, \$
<i>E</i>		Endable	id, num	\$

Parsing Algorithms

LL(1)	Not LL(1) — it contains a first set clash.	Parsing table
LR(0)	Not LR(0) — it contains a reduce-reduce conflict.	Automaton, Parsing table
SLR(1)	Not SLR(1) — it contains a reduce-reduce conflict.	Parsing table
LR(1)	The grammar is LR(1).	Automaton, Parsing table
LALR(1)	The grammar is LALR(1).	Automaton, Parsing table

Zdroj vzorových gramatik : <http://smlweb.cpsc.ucalgary.ca/start.html>

Nová verze prostředí: <http://mdaines.github.io/grammophone/#>

Příklady na automatické testy gramatik

$S \rightarrow E.$

$E \rightarrow T E1.$

$E1 \rightarrow \text{plus } T E1$

$\quad | \text{ minus } T E1$

$\quad | .$



epsilon

$T \rightarrow F T1.$

$T1 \rightarrow \text{mult } F T1$

$\quad | \text{ divide } F T1$

$\quad | .$

$F \rightarrow \text{number}$

$\quad | \text{id} .$

$A \rightarrow B C b$

$\quad | a B .$

$B \rightarrow b$

$\quad | .$

$C \rightarrow c A$

$\quad | .$

Výsledek analýzy LL gramatiky pro aritmetické výrazy

EditTransformAnalyze

Type a grammar here:

```
E -> + E | E  
E' -> + E' | .  
T -> ( E ) | int  
F -> ( E ) | int
```

Analysis

Sanity Checks

- All nonterminals are reachable.
- All nonterminals are realizable.
- The grammar contains no cycles.
- The grammar is **null unambiguous**.

Example Sentences

- int
- (E)
- int + int
- int + (E)
- int * int
- int * (E)
- (E) + int
- (E) + (E)
- (E) * int
- (E) * (E)

[More example sentences](#)

Nonterminals

Symbol	Nullable?	Endable?	First set	Follow set
E		Endable	(E), int	\$
E'	Nullable	Endable	+	\$
T		Endable	(E), int	+, \$
T'	Nullable	Endable	*	+, \$
F		Endable	(E), int	+, *, \$

Parsing Algorithms

LL(1)	The grammar is LL(1).	Parsing table
LR(0)	Not LR(0) — it contains a shift-reduce conflict.	Automaton , Parsing table
SLR(1)	The grammar is SLR(1).	Parsing table
LR(1)	The grammar is LR(1).	Automaton , Parsing table
LALR(1)	The grammar is LALR(1).	Automaton , Parsing table